



Unidad 3 Clase 7

Distintos modelos NoSQL

MODELO TABULAR Y CQL

Diseñar estructuras de datos orientadas a consultas en entornos distribuidos, utilizando CQL para comprender el manejo de eventos masivos, series temporales y los compromisos entre escalabilidad y consistencia.

Bases de Datos de Columna Ancha (Modelo Tabular)

Flexibilidad y escalabilidad para grandes volúmenes de datos

Las bases de datos de columna ancha (wide-column o column family) son un modelo NoSQL diseñado para almacenar información masiva de forma distribuida. No deben confundirse con una base de datos relacional tradicional.

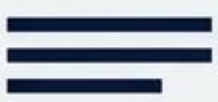
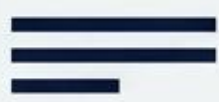
- **Estructura Dinámica**

- A diferencia de SQL (esquema rígido), cada fila puede tener una configuración completamente diferente de columnas.

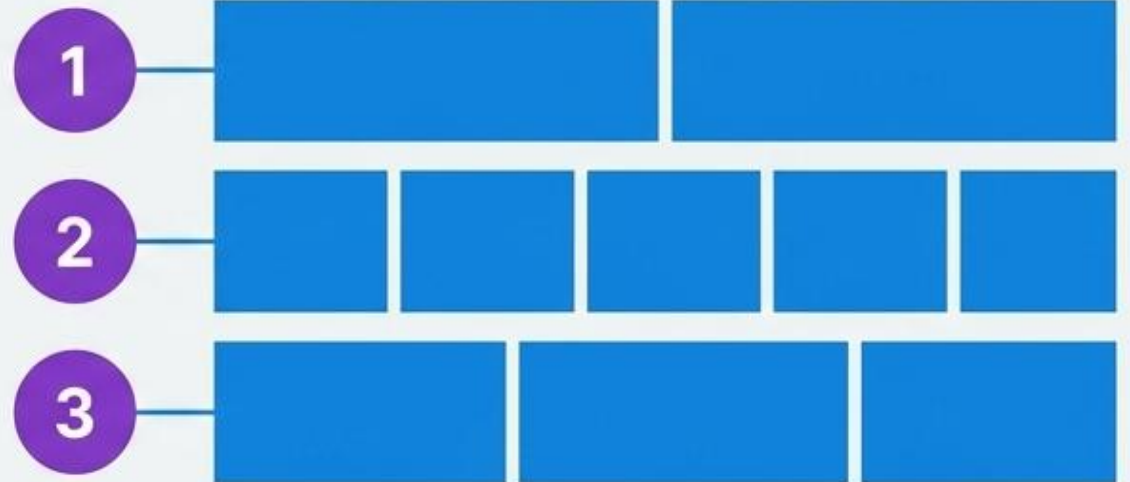
- **Orientación**

- Su diseño no prioriza relaciones perfectas entre entidades, sino distribuir datos eficientemente a través de múltiples nodos.

Modelo Relacional (SQL)

		NULL	
	NULL		
			NULL

Modelo Tabular (Wide-column)

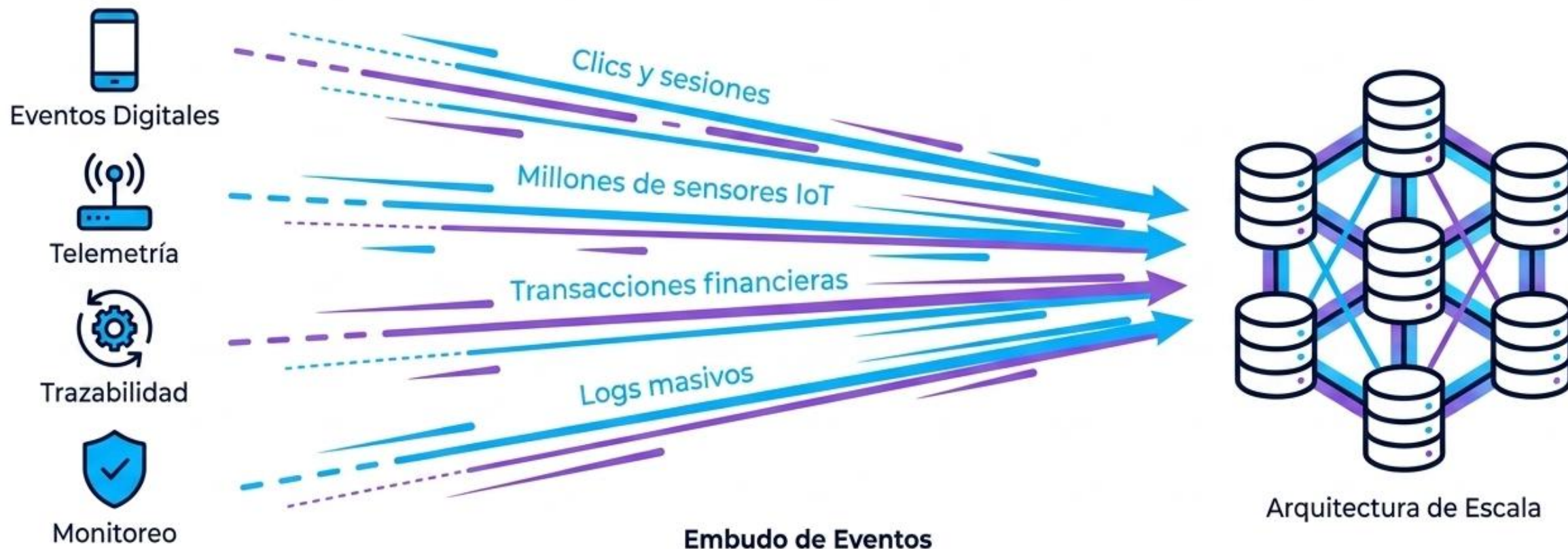


¿Qué problema resuelven?

Diseñadas para la era de los datos continuos y masivos



Este modelo surge en escenarios donde el modelo relacional se vuelve ineficiente y excesivamente costoso: entornos de altísimo volumen, velocidad de escritura extrema y sistemas distribuidos.





Cuándo elegir el modelo tabular

Evaluando las fortalezas y los límites del entorno

✓ ADECUADO PARA

- Escalabilidad horizontal masiva.
- Alta disponibilidad ininterrumpida.
- Cargas de escritura intensivas (flujos/streams).
- Patrones de consultas conocidos de antemano.

✗ NO ADECUADO PARA

- ✗ Transacciones ACID estrictas para lectura/escritura.
- ✗ Consultas exploratorias o libres sobre cualquier campo.
- ✗ Modelos dependientes de cruces de tablas (Joins).
- ✗ Agregaciones dinámicas en el motor (como SUM o AVG).

Comparación Conceptual

Relacional (SQL) vs. Tabular (Wide-column)

Dimensión	Modelo Relacional (RDBMS)	Modelo Tabular (Wide-column)
Estructura	Fija y estandarizada. Filas uniformes.	Dinámica. Columnas variables por fila.
Relaciones	Entidades vinculadas por Foreign Keys y Joins.	Entidades desnormalizadas. Sin Joins nativos.
Distribución	Centralizada (Scale-up / Vertical).	Nativamente distribuida (Scale-out / Horizontal).
Consultas	Flexibles (filtrado por cualquier campo).	Restringidas (optimizadas por claves predefinidas).

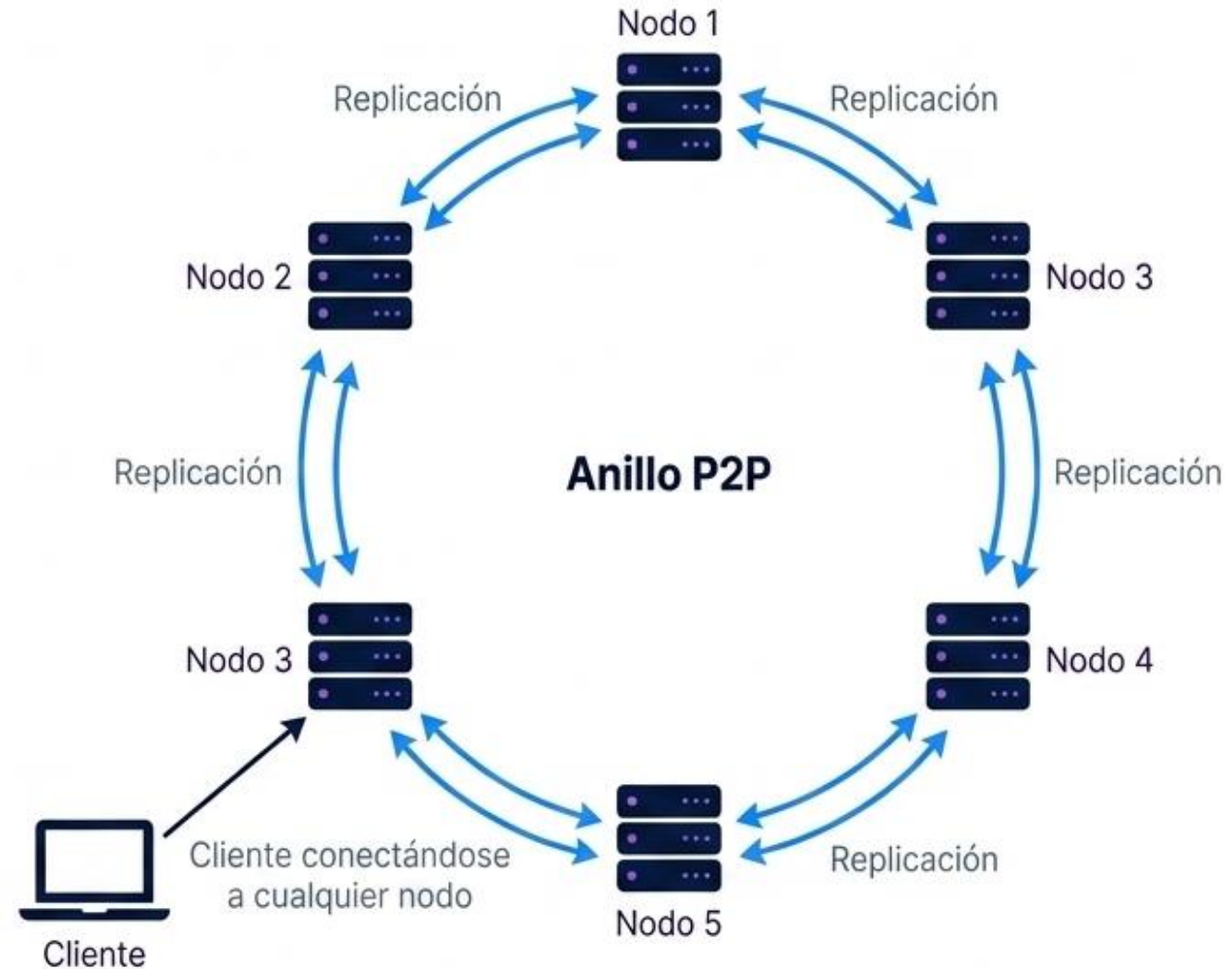
Apache Cassandra: Referencia en Datos Distribuidos

El estándar open-source para el modelo tabular

Apache Cassandra es una base de datos NoSQL de columna ancha, distribuida y altamente escalable. Nacida para resolver las limitaciones del modelo relacional a escala global.

Arquitectura Base:

- **Clúster y Nodos:** El sistema se compone de múltiples servidores independientes (nodos) agrupados lógicamente (clúster).
- **Escalabilidad Horizontal:** Su diseño permite manejar desde gigabytes hasta petabytes simplemente sumando nuevos nodos al anillo, aumentando la capacidad de forma lineal.





Arquitectura de Escala

Diseñada para no fallar y escalar infinitamente



Arquitectura sin maestro (P2P)

Protocolo Peer-to-Peer. Cualquier nodo actúa como coordinador y atiende lecturas/escrituras. No hay cuellos de botella.



Alta Disponibilidad

Si un nodo falla, el clúster sigue operando con normalidad. No hay un punto único de fallo (SPOF).



Tolerancia a fallos

Los datos se replican automáticamente en múltiples nodos según las estrategias configuradas.



Escrituras hiperrápidas

Diseñada estructuralmente para que la inyección de datos (escritura) sea computacionalmente barata y masiva.

Cuándo Cassandra NO es la mejor opción

Comprendiendo los compromisos tecnológicos (Trade-offs)

Cassandra es una herramienta especializada de alto rendimiento, pero no es una base de datos de propósito general. Sus características de diseño imponen límites claros:



No soporta Joins complejos: Cruzar información en tiempo real entre millones de registros distribuidos en distintos servidores físicos es inviable.



Imposibilidad de consultas libres: Filtrar por una columna que no es clave genera un escaneo masivo de todo el clúster (ineficiencia extrema).



Sin transacciones relacionales: Utiliza un modelo de consistencia eventual; no asegura el bloqueo transaccional estricto tradicional.



Agregaciones complejas delegadas: Cálculos ad-hoc en tiempo real deben realizarse en la capa de la aplicación, no en el motor de lectura rápida.



El Diseño Orientado a la Consulta

El cambio de paradigma en el modelado tabular

Para aprovechar el poder del modelo distribuido ante series temporales y eventos masivos, debemos invertir el proceso de diseño relacional. Desnormalizar y duplicar la información es la estrategia correcta para evitar lecturas costosas.



“*No se diseña primero la entidad; se diseña **primero la consulta** que debe resolverse a escala.*”

Arquitectura General de Cassandra

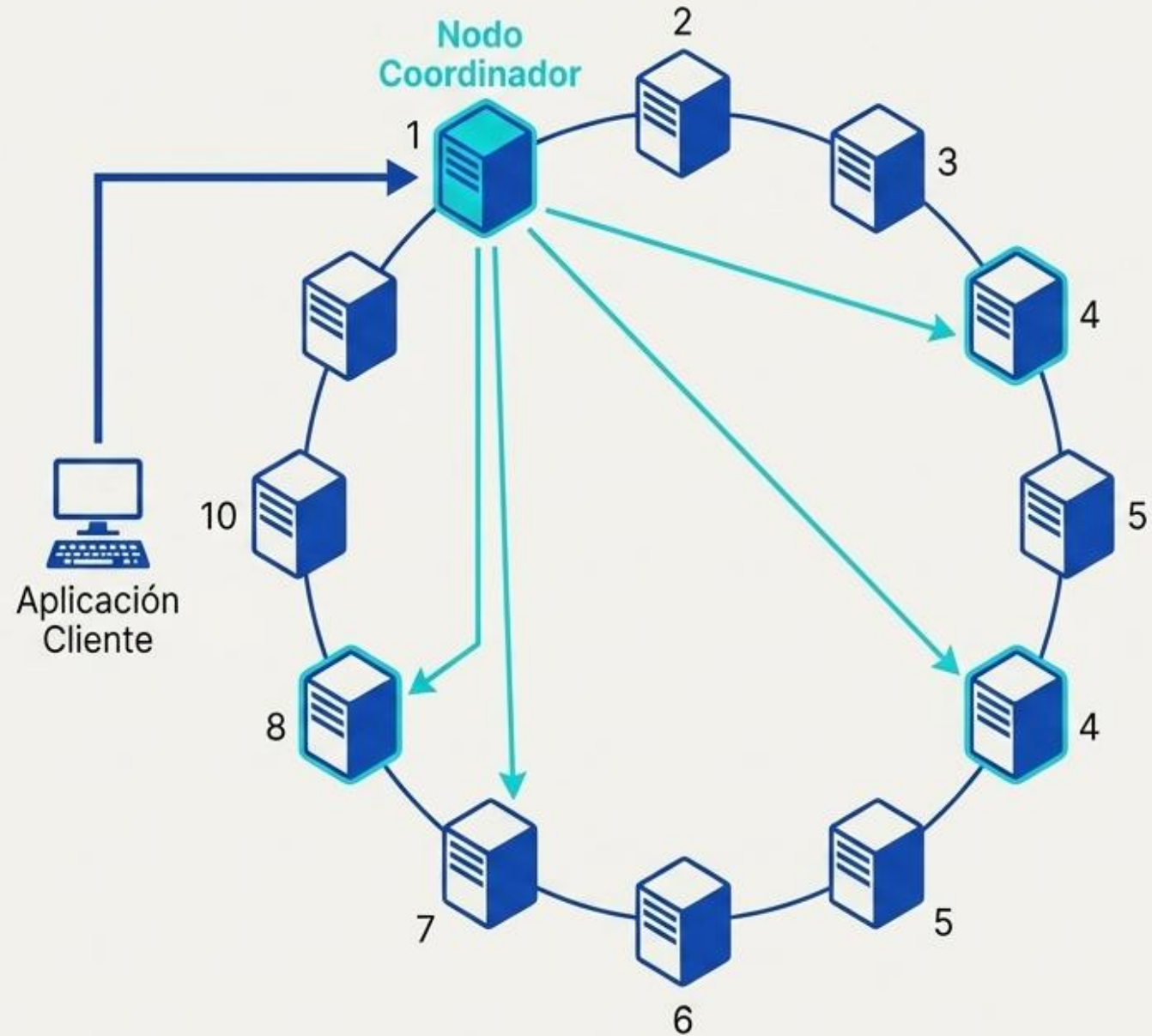
Cassandra utiliza una arquitectura distribuida peer-to-peer (P2P) en forma de anillo, diseñada para eliminar cuellos de botella y puntos únicos de fallo.

Componentes clave:

- Nodo: Instancia individual donde se almacenan físicamente los datos.
- Clúster: Agrupación de múltiples nodos (puede abarcar varios Data Centers).

El rol del Nodo Coordinador:

- No existen nodos 'maestros'. Todos son exactamente iguales.
- La aplicación cliente puede conectarse a cualquier nodo.
- El nodo que recibe la petición actúa como Coordinador, ruteando la operación hacia los nodos exactos que contienen los datos.



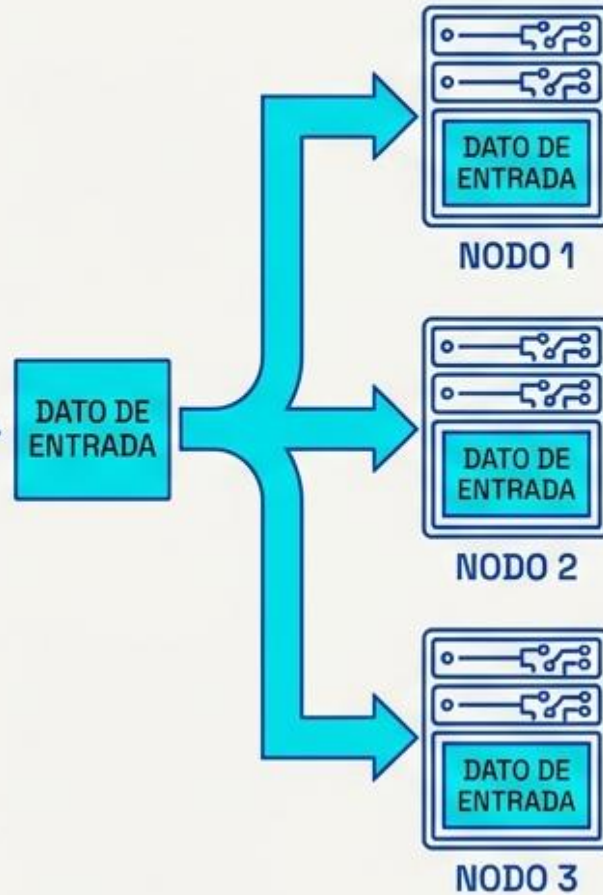
REPLICACIÓN Y ALTA DISPONIBILIDAD

RESILIENCIA ACTIVA

Para garantizar que la información no se pierda, se implementa un sistema de replicación automática.

- **Tolerancia a fallos:** Si un servidor físico cae, el clúster sigue operando sin interrupciones mediante las réplicas.
- **Alta disponibilidad:** Múltiples nodos pueden responder en paralelo a la misma consulta.

PASO 1: REPLICACIÓN INICIAL



PASO 2: FALLO DEL SERVIDOR



PASO 3: RECUPERACIÓN Y ALTA DISPONIBILIDAD



CONSISTENCIA EVENTUAL

Al mantener copias distribuidas en red, existen fracciones de segundo de desincronización.

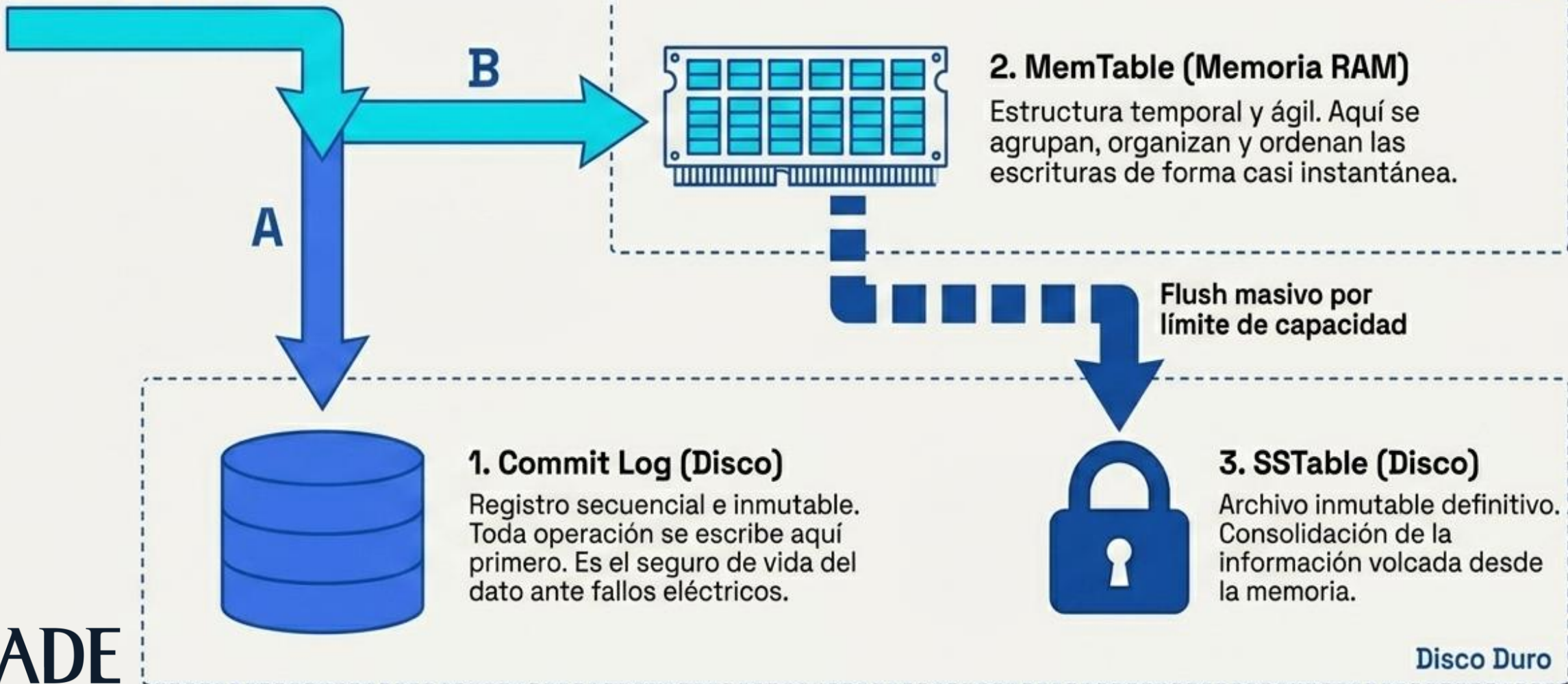
- El sistema prioriza la disponibilidad total sobre la sincronización instantánea absoluta.
- Los datos convergerán a su estado final correcto (Eventual Consistency) gracias a protocolos de reparación en segundo plano.



MECÁNICA INTERNA: EL CAMINO DE LOS DATOS

El proceso de escritura dentro de un nodo único involucra tres estructuras operando en tándem para garantizar persistencia y velocidad:

PETICIÓN DE ESCRITURA



POR QUÉ CASSANDRA ES IDEAL PARA EVENTOS MASIVOS

La arquitectura de memoria y disco genera una asimetría clave en el rendimiento del motor:

Escrituras Económicas (Ultra-Rápidas)



- Escribir en RAM (MemTable) no tiene latencia física.
- Escribir en disco (Commit Log) es puramente secuencial (Append-only), evitando el desgaste de cabezales.

VEREDICTO: Ideal para volúmenes masivos y constantes de eventos (IoT, logs, clics, telemetría).

Lecturas Costosas (Requieren Optimización)



- Para leer un dato, el motor debe buscar en la MemTable y rastrear múltiples archivos SSTables históricos.
- Requiere consolidar fragmentos esparcidos físicamente.

VEREDICTO: Exige compactación periódica y, fundamentalmente, un diseño de tabla perfecto.

EL CAMBIO DE MENTALIDAD: NORMALIZAR VS. CONSULTAR

El enfoque relacional colapsa en entornos distribuidos porque los JOINS masivos destruyen la red. Cassandra exige invertir el flujo de diseño.

PARADIGMA RELACIONAL (SQL)



PARADIGMA TABULAR (CQL)



DISEÑO ORIENTADO A CONSULTAS (QUERY-DRIVEN MODELING)

En Cassandra nunca se diseña una tabla "general" para guardar todo un dominio. Se diseña a partir de las preguntas exactas que el sistema debe responder.

¿Cuáles son los últimos eventos de un usuario por día?



Tabla 1

¿Cuáles fueron los eventos sospechosos de una fecha concreta?



Tabla 2

¿Cuántos intentos fallidos tuvo un usuario en una ventana temporal?



Tabla 3

¿Qué eventos ocurrieron por tipo y por fecha?



Tabla 4

1 PREGUNTA = 1 TABLA

Cada pregunta origina una tabla física independiente, estructurada para entregar su respuesta en una sola pasada de lectura.

La Desnormalización como Estrategia, no como Error

Si un evento único debe responder a varias de las preguntas de la arquitectura, ¿qué hacemos?



(En SQL)

Se guarda una sola vez en una tabla maestra.
Se usan costosos JOINS en tiempo real para armar la vista.

(En Cassandra)

El mismo evento se escribe repetidas veces en múltiples tablas simultáneamente. Gastamos disco barato hoy, para garantizar una lectura instantánea mañana.

Ejemplo Conceptual: De la Necesidad a la Estructura

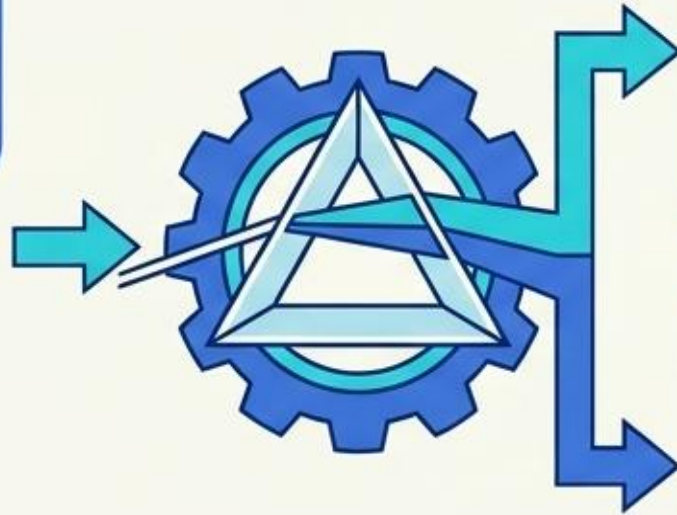
Cómo se traduce el flujo de pensamiento tabular a la práctica física:

1. La Necesidad (El Input)

Ver eventos de un usuario específico en un día exacto, ordenados del más reciente al más antiguo.



2. Ingeniería de Claves (El Mecanismo)



Clave de Partición (El Dónde)

usuario_id + fecha.
Define en qué nodo físico se agrupa.

Clave de Ordenamiento (El Cómo)

evento_ts. Fuerza mecánicamente el orden cronológico descendente en el disco.



3. La Tabla Resultante (El Output)

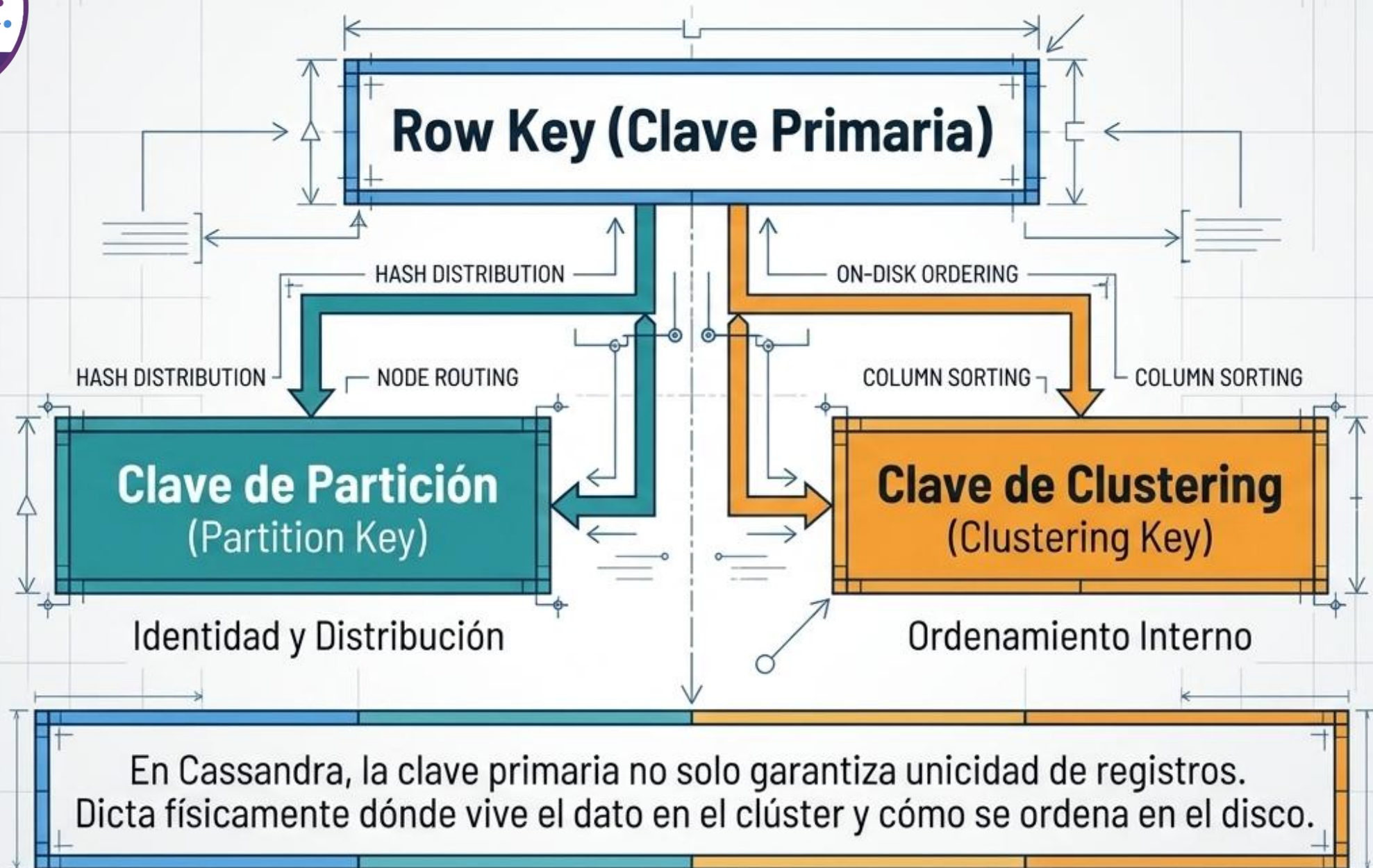


eventos_por_usuario_dia					

Cassandra salta directamente al nodo del usuario y devuelve un bloque que ya está pre-ordenado. Cero cálculos en ejecución.

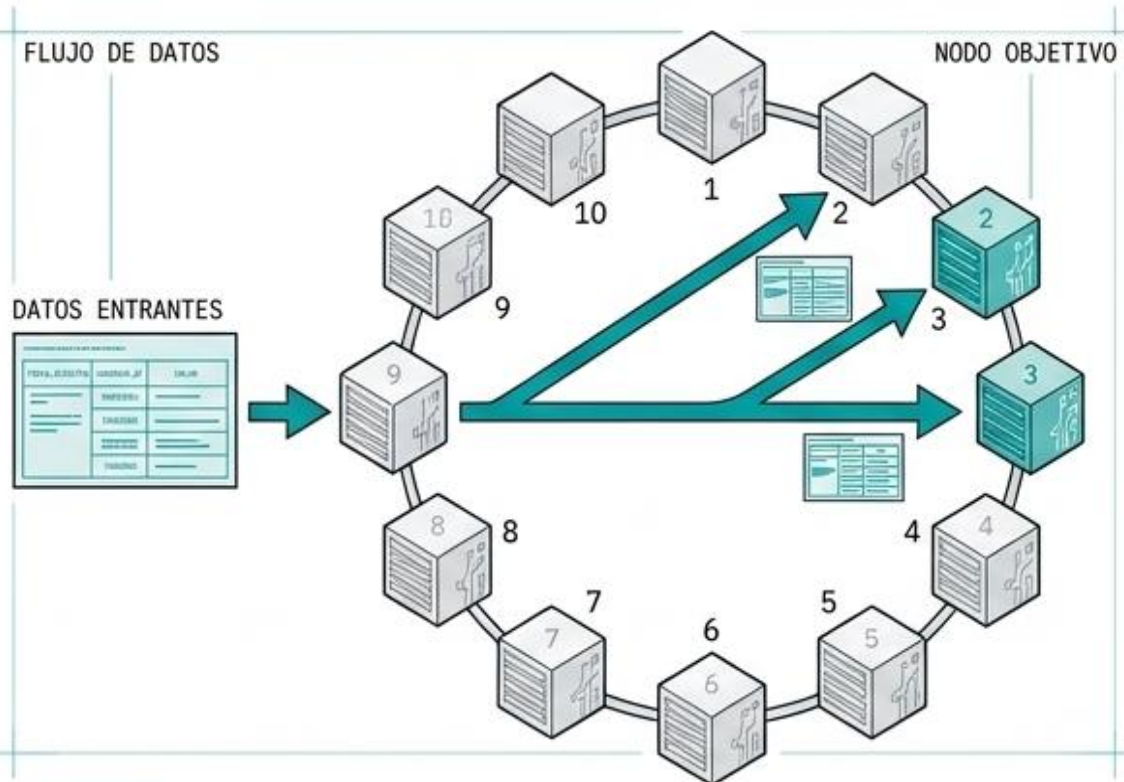


La Anatomía de la Clave Primaria



Dos Fuerzas Estructurales: Partición y Clustering

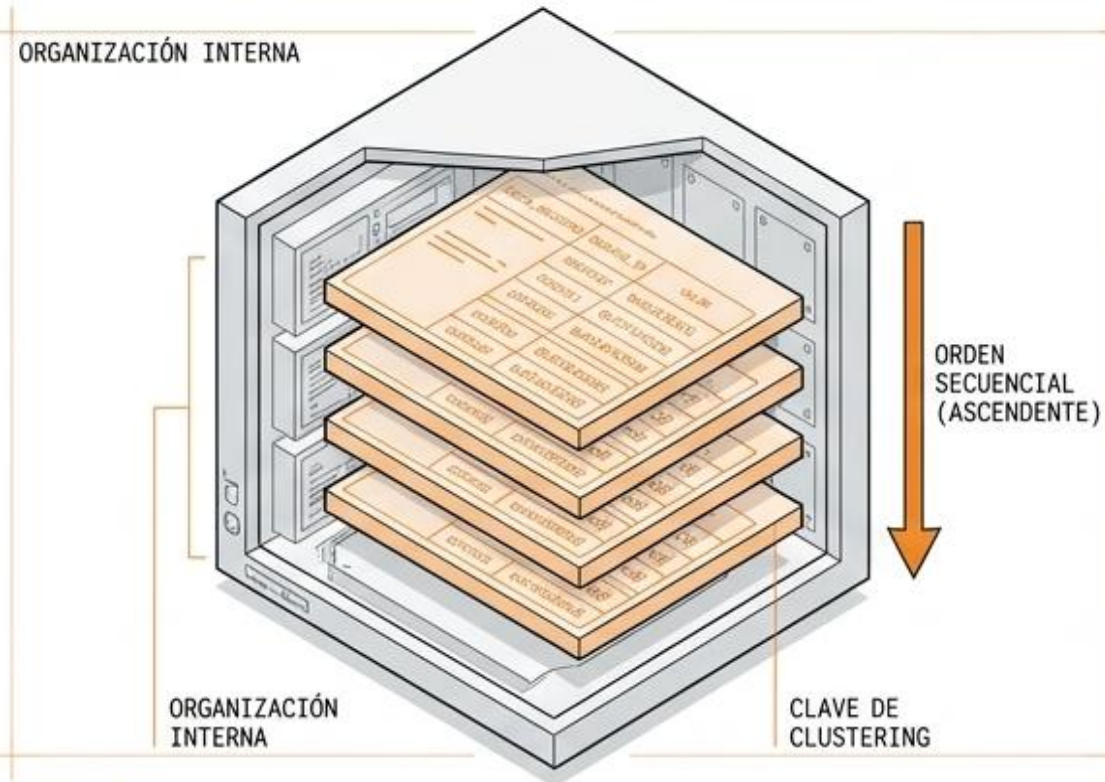
¿Dónde va y cómo se agrupa?



STRUCTURAL

La partición agrupa datos relacionados y los envía al nodo correcto.

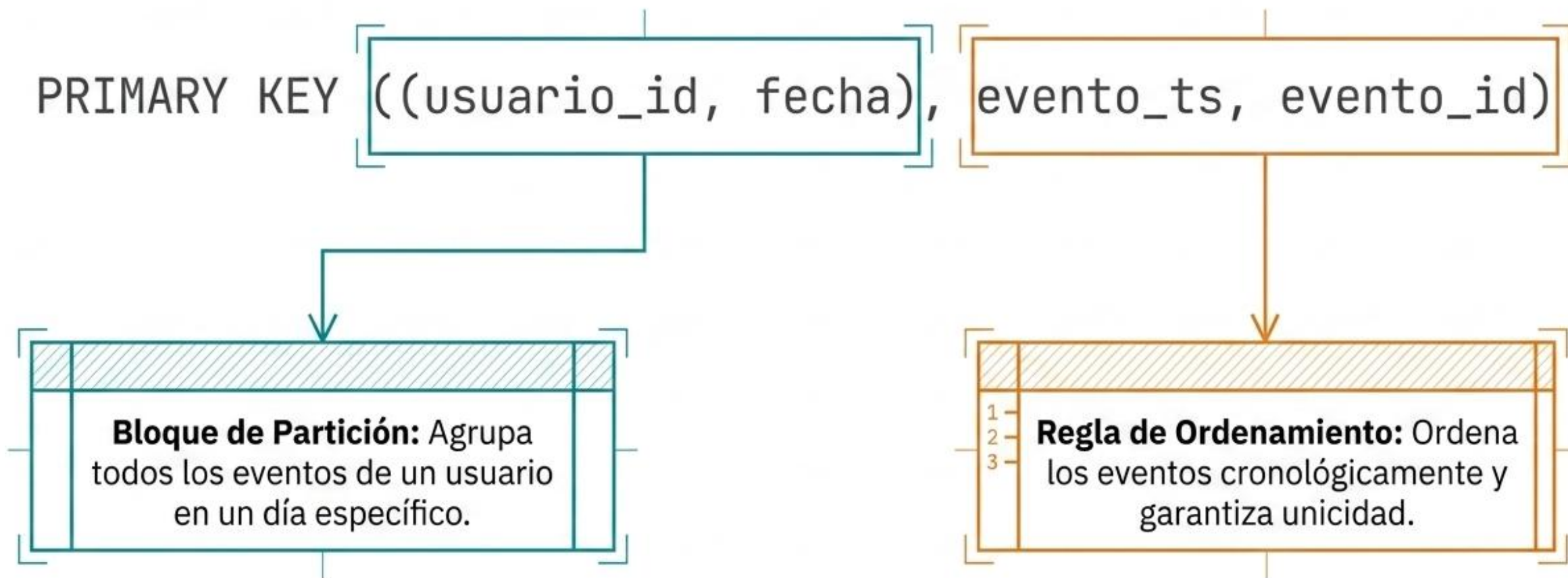
¿Cómo se organiza una vez allí?



STRUCTURAL

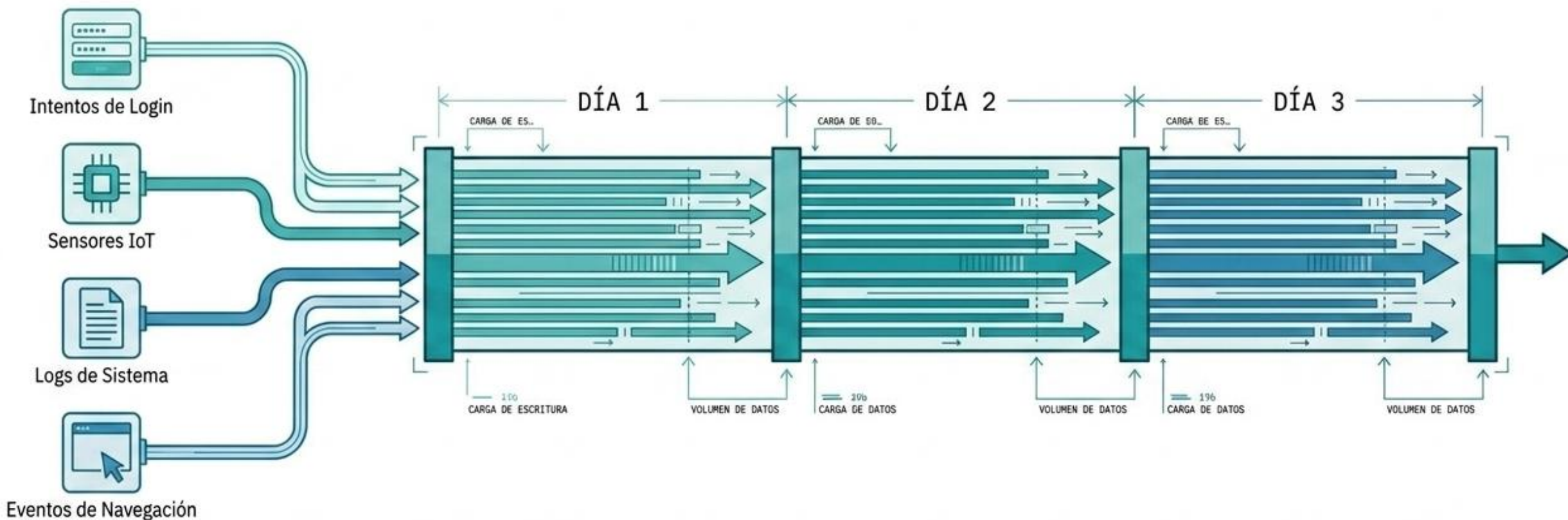
El clustering define el orden secuencial de esos datos dentro de la partición.

Descomponiendo una Clave Compuesta



Los paréntesis dobles ((...)) definen explícitamente qué atributos conforman la ruta principal de acceso a los datos.

El Tiempo como Eje Estructural



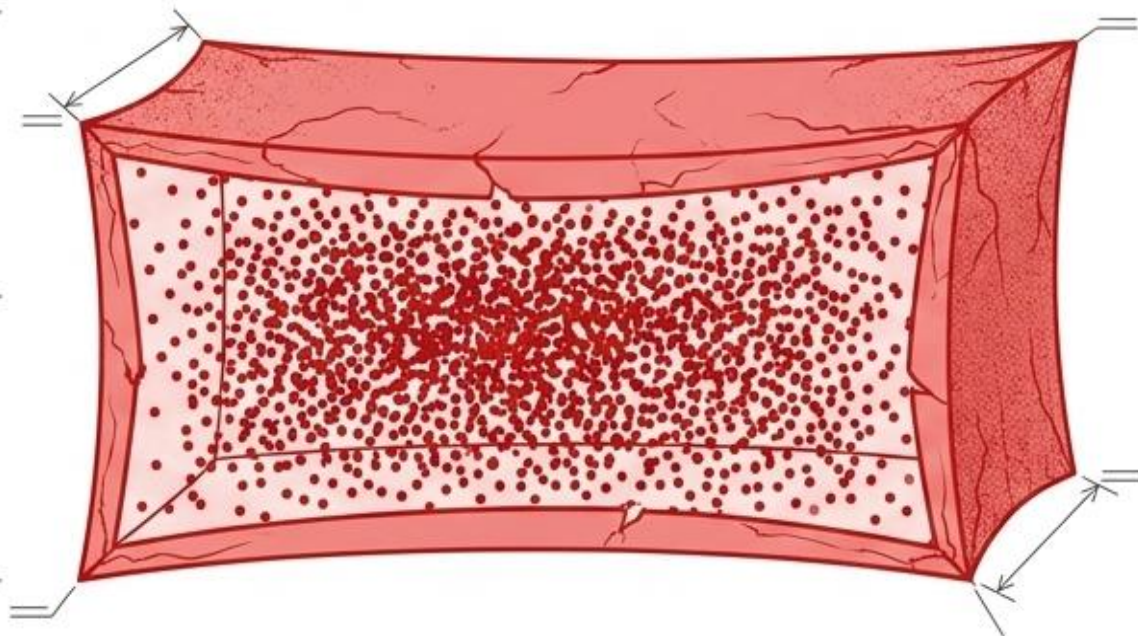
STRUCTURAL

En escenarios de eventos masivos, el tiempo no es simplemente un atributo descriptivo.
Es el criterio estructural primario para distribuir la carga de escritura y segmentar las lecturas de forma predecible.

El Arte de Particionar: Tamaño y Distribución

Bad Design (Diseño Inadecuado)

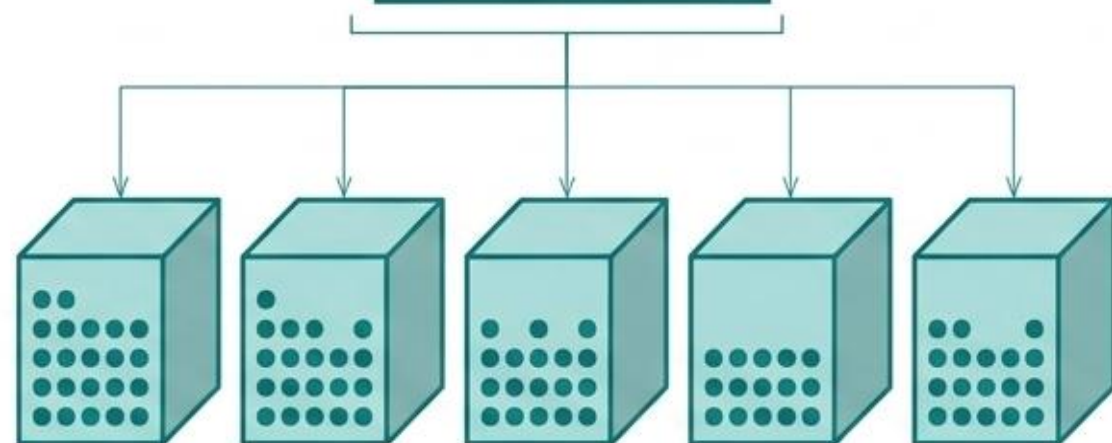
PRIMARY KEY (usuario_id, evento_ts)



Mala Idea: Particiones Gigantes. Agrupar todo el historial de un usuario crea cuellos de botella.

Good Design (Diseño Óptimo)

PRIMARY KEY ((usuario_id, fecha), evento_ts)



Mejor Idea: Particiones Acotadas. Dividir por períodos razonables asegura un rendimiento estable y predecible.

El diseño debe evitar **particiones masivas** que concentren demasiados datos, y particiones microscópicas que fueren la lectura en múltiples nodos.

CQL: Sintaxis Familiar, Lógica Diferente

Estructura / DDL

CREATE KEYSPACE

USE

CREATE TABLE

Operación / DML

INSERT

SELECT

UPDATE

DELETE

ATENCIÓN: CQL se lee como SQL, pero no se comporta como SQL. Cassandra no soporta **JOINS**, carece de **integridad referencial** estricta y penaliza severamente el **filtrado libre**.

Las sentencias deben diseñarse en función de la arquitectura distribuida, no del modelo relacional.

Diseño Orientado a Consultas en Acción

DDL - Esquema de la Tabla

```
CREATE TABLE eventos_por_usuario_dia (  
  ...  
  PRIMARY KEY ((usuario_id, fecha), evento_ts)  
);
```

DML - Consulta Eficiente

```
SELECT * FROM eventos_por_usuario_dia  
WHERE usuario_id = 'USR1001' AND fecha = '2026-05-03';
```

Una consulta es **inmediata y eficiente** cuando sus filtros coinciden exactamente con la Clave de Partición.

El Límite del Modelo: Consultas Válidas vs. Problemáticas



```
SELECT * FROM eventos_por_usuario_dia  
WHERE usuario_id = 'USR1001' AND fecha = '2026-05-03'
```

Válida: Utiliza la ruta de la clave de partición completa.



```
SELECT * FROM eventos_por_usuario_dia WHERE riesgo >= 80
```

Problemática: Intenta filtrar por una columna fuera de la clave.



ALLOW FILTERING

Advertencia: Es un bypass de emergencia que escanea todo el clúster. No es una solución de diseño.

SÍNTESIS: En Cassandra, una buena tabla no es la que guarda todos los datos posibles, sino la que responde eficientemente una consulta importante.



PROXIMA CLASE



Escalabilidad, replicación y teorema CAP en NoSQL

Tema:

Administración y recuperación de datos NoSQL.

Interfaces de administración.

Escalamiento horizontal.

Replicación y particionamiento (profundización).

Teorema CAP aplicado a Mongo, Cassandra y Redis.
